

Design of a Simple RISC Processor – Full Project Report

CSC 320 — Computer Organization

Lebanese American University

Fall 2025

1. Introduction

This report presents the complete design of a simple RISC processor as required by the course project. The processor supports arithmetic operations, shift operations, and specialized memory access instructions involving shifted register offsets. It follows a four-stage control flow: **Fetch**, **Decode**, **Execute**, and **Write-Back**, and uses separate **instruction memory** and **data memory**.

The processor contains **8 general-purpose registers (R0–R7)** and a **Program Counter (PC)**. All instructions are encoded into a **16-bit instruction word**.

2. Register Identifiers (Q1)

The processor includes 8 registers: R0–R7. Each register is uniquely identified using a 3-bit binary code.

Register	Binary Identifier
R0	000
R1	001
R2	010
R3	011
R4	100
R5	101
R6	110
R7	111

The PC register is not encoded in instruction fields and is managed separately.

3. Instruction Size (Q2)

Chosen instruction size: 16 bits

This size is the smallest power-of-two value that can hold all required fields.

Bit Breakdown

- **Opcode:** 3 bits (8 instructions $\rightarrow 2^3 = 8$)
- **Destination register:** 3 bits
- **Source/Base register:** 3 bits
- **Offset register:** 3 bits
- **Shift amount:** 4 bits

Total = $3 + 3 + 3 + 3 + 4 = 16$ bits

This makes instruction memory word-addressable and simplifies PC incrementing.

4. Register Size (Q3)

All registers are **32 bits wide**.

Justification: - Provides compatibility with 32-bit word operations. - Supports large data memory addressing. - Matches typical RISC educational architectures.

5. Maximum Data Memory Capacity (Q4)

Data memory is **word-addressable**, with each word = **4 bytes**.

Since addresses come from 32-bit registers: - Maximum number of addressable words = 2^{32} - Total size = 2^{32} words $\times$ 4 bytes/word = 2^{34} bytes = **16 GB**

Thus, the processor can theoretically support up to **16 gigabytes of data memory**.

6. Instruction Structure and Field Sizes (Q5)

The processor uses **two formats**, both 16 bits wide.

6.1 Format A — ALU Instructions

```
|15..13|12..10|9..7 |6..4 |3..0 |
|Opcode|Rdest |Rsrc1|Rsrc2|Sh    |
```

Used for: - ADD - SUB - SHL - SHR

Fields: - Opcode (3 bits) - Rdest (3 bits) - Rsrc1 (3 bits) - Rsrc2 (3 bits) - Sh (4 bits) — immediate shift amount

6.2 Format B — Memory Instructions (Load/Store with shifted offset)

```
|15..13|12..10|9..7 |6..4 |3..0 |
|Opcode|Rdest |Rbase|Roff |Sh    |
```

Used for: - ShiftLeftLoad - ShiftRightLoad - ShiftLeftStore - ShiftRightStore

Fields: - Opcode (3 bits) - Rdest (3 bits) → destination for loads, source for stores - Rbase (3 bits) - Roff (3 bits) → offset register - Sh (4 bits) → shift amount

Address calculation:

```
Address = Rbase + (Roff << Sh)    for left-shift instructions
Address = Rbase + (Roff >> Sh)    for right-shift instructions
```

7. Instruction Syntax & Opcodes (Q6)

The processor supports **eight instructions**, each with a dedicated opcode.

Opcode	Binary	Instruction	Syntax	Function
0	000	ADD	ADD Rd, Rs1, Rs2	Rd = Rs1 + Rs2
1	001	SUB	SUB Rd, Rs1, Rs2	Rd = Rs1 - Rs2
2	010	SHL	SHL Rd, Rs, Sh	Rd = Rs << Sh
3	011	SHR	SHR Rd, Rs, Sh	Rd = Rs >> Sh
4	100	ShiftLeftLoad	LL Rd, Rbase, Roff, Sh	Rd = MEM[Rbase + (Roff << Sh)]
5	101	ShiftRightLoad	LR Rd, Rbase, Roff, Sh	Rd = MEM[Rbase + (Roff >> Sh)]
6	110	ShiftLeftStore	SL Rs, Rbase, Roff, Sh	MEM[...] = Rs
7	111	ShiftRightStore	SR Rs, Rbase, Roff, Sh	MEM[...] = Rs

8. Program Counter Increment (Q7)

Instruction memory is **word-addressable**, and each instruction is **1 word (16 bits)**.

Therefore:

PC = PC + 1 after every instruction fetch.

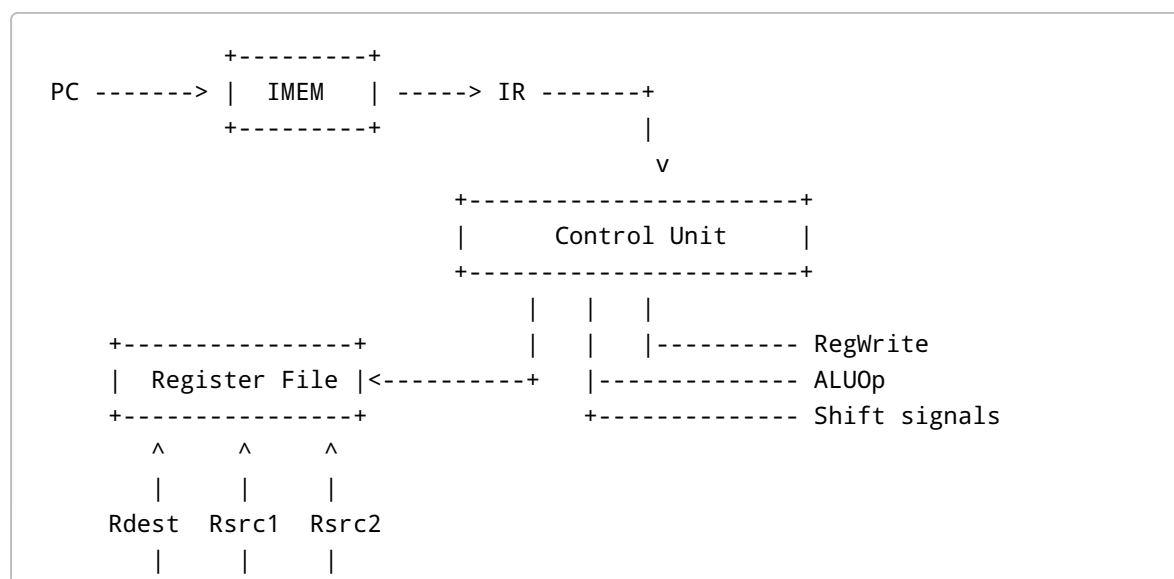
9. Datapath Design (Q8)

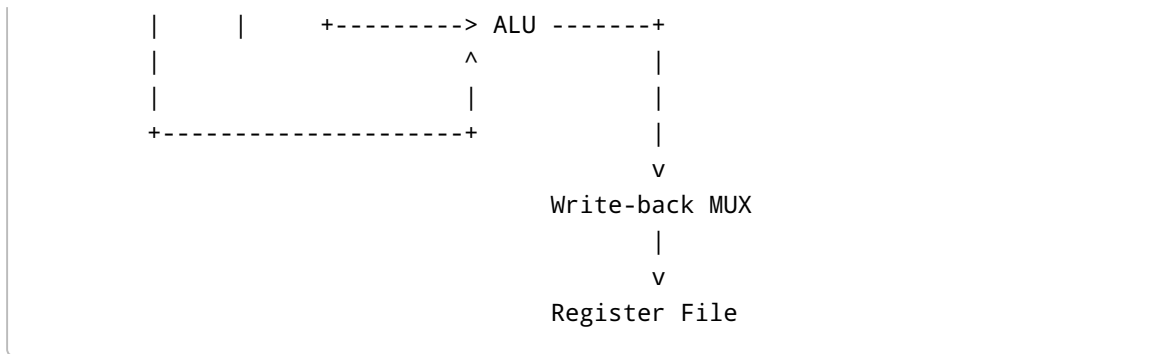
Below are text-based block diagrams representing the datapath. These correspond to the required processor behavior.

9.1 Datapath Components

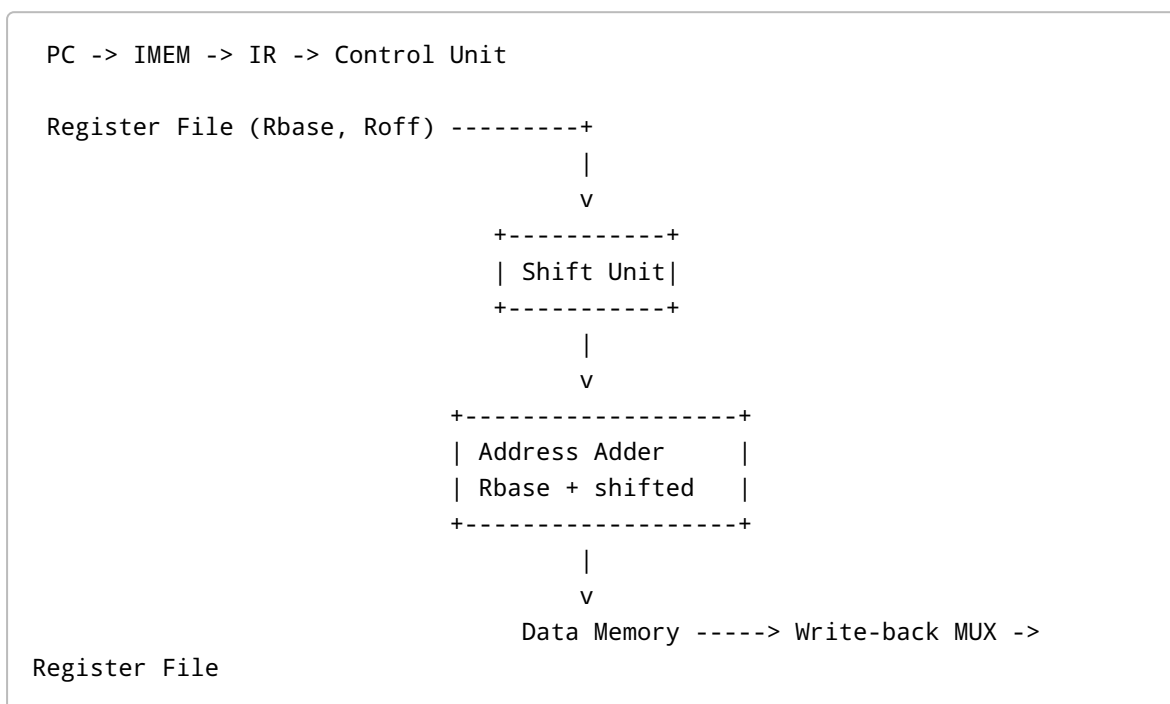
- **Program Counter (PC)**
 - **Instruction Memory (IMEM)**
 - **Instruction Register (IR)**
 - **Register File (R0–R7)**
 - **Shift Unit** (left/right shift by immediate)
 - **ALU** (add/sub)
 - **Address Adder** (for memory instructions)
 - **Data Memory (DMEM)**
 - **Write-back MUX**
 - **Control Unit**
 - **Various multiplexers**
-

9.2 Datapath for ALU Instructions

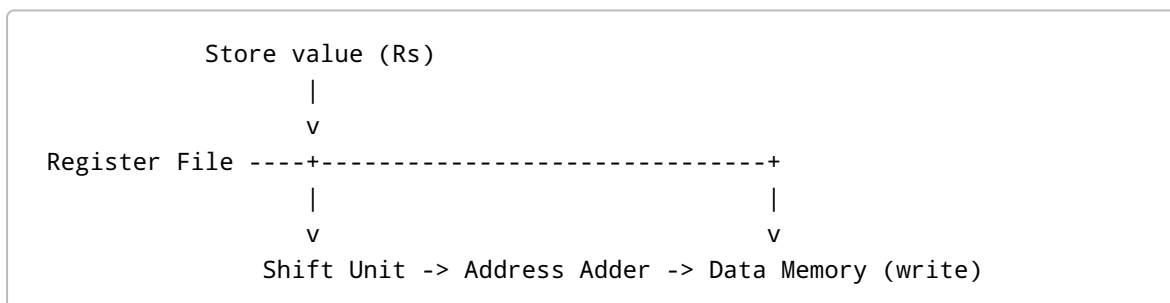




9.3 Datapath for Load Instructions



9.4 Datapath for Store Instructions



10. Control Flow Stages

Each instruction goes through:

1. Fetch

- $IR \leftarrow IMEM[PC]$
- $PC \leftarrow PC + 1$

2. Decode

- Opcode is decoded
- Register file outputs Rdest, Rsrc1, Rsrc2 OR Rbase, Roff
- Shift amount extracted

3. Execute

- ALU operations for arithmetic
- Shift + address addition for memory instructions

4. Write-Back

- ALU or DMEM output is written into Rdest (for loads)
 - Stores do not write to registers
-

11. Example Instruction Executions

11.1 Example: LL R1, R2, R3, 2

```
Address = R2 + (R3 << 2)
R1 = MEM[Address]
```

11.2 Example: ADD R4, R1, R2

```
R4 = R1 + R2
```

12. Demo Requirements (for submission)

Your demo must show: - Datapath with highlighted signal values - Flow of at least one ALU and one memory instruction - Example of shifted offset calculation - Register file before/after execution

13. Conclusion

This report fully defines the processor architecture, instruction formats, opcodes, datapath, and execution behavior. It meets all requirements outlined in the CO Project (Fall 2025) assignment.

End of Report